

Laboratorio II

Corso A

Lezione 4

- Programmazione C di base
 - Comandi condizionali e iterativi
 - Tipi e conversioni fra tipi
 - Array
 - Parametri dalla linea di comando

Comandi condizionali

- `espressione` rappresenta qualunque espressione numerica
- Se `espressione` ha valore uguale a `0` (***false***) allora viene eseguito il primo comando
- Altrimenti, viene eseguito il secondo comando
- Esiste anche nel formato senza `else`
- Sono annidabili
 - Per evitare il **dangling else**, viene utilizzato un blocco di codice racchiuso tra `{ }`

If-else:

```
if (espressione)
    comando
else
    comando
```



Comandi iterativi

- Rappresentano la possibilità di ripetere un blocco di codice
- `espressione` rappresenta qualunque espressione numerica
- Se `espressione` ha valore diverso da 0 (**vero**) allora viene eseguito il comando (corpo del ciclo) e si torna all'inizio del ciclo
- Altrimenti, si prosegue dopo il comando `while`
- **ATTENZIONE: può non terminare mai!**

```
#include <stdio.h>

int main(void) {

    int n= 12345;

    while (n>0){
        printf("%d\n",n%10);
        n/=10;
    }

    return 0;
}
```

DO- WHILE

do

comando

while (espressione)

WHILE

while (espressione)

comando

Comandi iterativi

- Viene valutata `espressione1`, una sola volta alla prima iterazione del ciclo `for`
- Se `espressione2` ha valore diverso da 0 (**vero**) allora viene eseguito il comando (corpo del ciclo)
- Ad ogni esecuzione di comando, al suo termine viene eseguita e valutata `espressione3`
- Equivalente al seguente ciclo `while`:

```
espressione1;
while(espressione2) {
    comando;
    espressione 3;
}
```

```
#include <stdio.h>

int main(void) {

    int n= 12345;

    for(;n>0;n/=10){
        printf("%d\\n",n%10);
    }

    return 0;
}
```

FOR

```
for (espressione1; espressione2; espressione3)
    comando
```



Istruzioni `break` e `continue`

- `break`
 - Causa l'uscita immediata dal ciclo che la contiene, o dall'istruzione `switch` in cui è presente

```
int a=0;
while(1) {
    if(a==10) break;
    a++;
}
```
 - Utile per uscire dai singoli blocchi di uno `switch`
- `continue`
 - Può essere contenuta solo in cicli `for`, `while` o `do-while`
 - Fa terminare l'iterazione del ciclo corrente e cominciare la successiva



Comandi condizionali

- Istruzione condizionale a scelta multipla
- `espressione` deve essere di tipo intero
- Le etichette devono essere uniche
- Il controllo passa all'espressione costante che corrisponde al valore di `espressione`
- Se non esiste, passa il controllo all'etichetta `default` (se esiste) o prosegue dopo lo `switch`
- **ATTENZIONE:** vengono eseguite tutte le istruzioni del costrutto `switch` in ordine a partire dall'etichetta corrispondente
 - Per evitare questo comportamento, devono essere inserite istruzioni `break` alla fine di ogni blocco

Switch:

```
switch (espressione){  
    case espr_costante:  
        comando  
        break;  
    case espr_costante:  
        comando  
        break;  
    default: comando  
}
```



Qualche funzione per I/O

- Libreria standard per I/O: `stdio`
- Per mostrare a video (stdout) dei valori - funzione `printf()`
 - `printf(stringa_formattazione)`
 - scrive la stringa passata come argomento.
 - la stringa di formattazione può contenere dei segnaposti (`%...`) per includere dei valori di variabili

```
#include <stdio.h>

#define LEN 20

struct Studente {
    char nome[LEN];
    int eta;
};

int main(void) {

    struct Studente s = { "Antonio",15};

    printf("Nome: %s, Età: %d\n",s.nome, s.eta);
    printf("Dimensione in memoria: %lu\n", sizeof(s));

    return 0;
}
```



Printf: segnaposti

- il segnaposto dipende dal tipo della variabile da scrivere
 - interi: `%d`, `%u` (unsigned). Si antepone `h` per `short` e `l` per `long`
 - reali: `%f`, `%e` (notazione scientifica), `%g` la piu' breve tra notazione standard e scientifica. Per i `long double` si antepone `L`
 - caratteri: `%c`
 - stringhe: `%s`
 - si possono includere opzioni di formattazione
 - ampiezza minima del campo `"%5d"`
 - numero di decimali dopo la virgola `"%.5f"`



Qualche funzione per I/O

- Per leggere da stdin - funzione `scanf()`
- Sintassi simile alla `printf`, prende una stringa di formattazione più gli **indirizzi (&)** delle variabili dove salvare i valori letti.
 - `scanf("%d %lf", &a, &b)` - legge due interi e li salva nelle variabili `a` e `b`
- I segnaposti sono gli stessi di `printf` tranne quello per double: `"%lf"`

```
#include <stdio.h>

int main(void) {
    int a=0;
    double b=0;
    scanf("%d %lf",&a,&b);
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

```
> ./main
```

```
7
```

```
9.8
```

```
a=7
```

```
b=9.800000
```

```
> 
```



Tipi di dato

- **C** è un linguaggio strongly typed
- Pochi tipi di dato primitivi:
 - `char`: valore intero corrispondente ad un singolo byte
 - `int`: numeri interi 123, -45, 0 (boolean), caratteri `'a'`, `'1'`
 - `float`: numeri reali 0.45, 23.0
 - `double`: numeri reali in precisione doppia
- Qualifier:
 - `short`, `long` per `int`: `short <= long`
 - `short int a = 1;`
 - `signed`, `unsigned` per `char` e `int`
 - `signed`: [-128, 127]
 - `unsigned`: [0, 255]
 - `long` per `double`
- Lunghezza della rappresentazione dipendente dalla macchina
- Si usa il seguente operatore unario (applicabile anche ad una espressione come `a+b`):
 - operatore `sizeof`

```
#include <stdio.h>

int main(void) {

    int a = sizeof(int);

    printf("%d\n", a);

    return 0;
}
```

Conversione tra tipi di dato

- **Implicita:**

- quando utilizziamo delle espressioni che coinvolgono tipi diversi
 - le variabili di tipo più 'corto' vengono trasformate in variabili di tipo più lungo
 - ***char, short -> int -> long -> float -> double -> long double***
- quando assegniamo un valore di una variabile ad un'altra variabile di tipo diverso
 - 'vince' il tipo della variabile a sinistra
 - perdita di informazione se a sinistra il tipo è più 'corto'

- **Esplicita**

- utilizziamo l'operatore di casting
 - Esempio:

```
long a;  
float b;  
b = (float) a;
```

- **Ci sono tipi di dato non compatibili: non possiamo trasformare una stringa in un intero**

- il compilatore ce lo dice : warning o errore

Conversione tra tipi di dato

```
#include <stdio.h>

int main(void) {
    int a=5;
    float b=a;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

```
#include <stdio.h>

int main(void) {
    int a=5;
    float b=a/2;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

Conversione tra tipi di dato

```
#include <stdio.h>

int main(void) {
    int a=5;
    float b=a;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

```
#include <stdio.h>

int main(void) {
    int a=5;
    float b=a/2;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

a=5
b=2.000000

Conversione tra tipi di dato

```
#include <stdio.h>

int main(void) {
    int a=5;
    float b=a;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

```
int main(void) {
    int a=5;
    float b=a/2.0;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

```
#include <stdio.h>

int main(void) {
    int a=5;
    float b=a/2;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

a=5
b=2.000000

```
int main(void) {
    int a=5;
    float b=(float)a/2;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

Conversione tra tipi di dato

```
#include <stdio.h>

int main(void) {
    int a=5;
    float b=a;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

```
#include <stdio.h>

int main(void) {
    int a=5;
    float b=a/2;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

a=5
b=2.000000

```
int main(void) {
    int a=5;
    float b=a/2.0;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

a=5
b=2.500000

```
int main(void) {
    int a=5;
    float b=(float)a/2;
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

a=5
b=2.500000

Array monodimensionali

- Collezione di valori **tutti** dello stesso tipo
- Per definire un array bisogna conoscere la sua dimensione (a meno che non usiamo i puntatori che vedremo in prossime lezioni)

```
int interi[8];
```

```
#define N 100
```

```
double reali[N];
```

```
float reali1[n]; //funziona solo se n è  
inizializzato
```

- Per accedere ad un elemento utilizziamo l'operatore [] :
a[i]
- Alla dichiarazione viene allocato uno **spazio contiguo** di dimensione giusta in memoria (sullo **stack**)

```
#include <stdio.h>

int main(void) {

    int n= 12345;
    int cifre[10]; //non molto bello :)
    int dim=0;

    for(;n!=0;dim++,n/=10){
        cifre[dim]=n%10;
    }

    for(int i=0;i<dim;i++){
        printf("%d\n",cifre[i]);
    }

    return 0;
}
```

Tipi di dato custom - struct

- Possiamo definire strutture di dati non omogenee
- Viene usata la parola chiave `struct`:
 - Definisce tipi di dati complessi con vari membri
 - **La parola chiave definisce un nuovo tipo, non dichiara una variabile!**
- L'ordine dei membri definisce l'ordine in memoria
- Lo spazio allocato ad una variabile definita di quel tipo è contiguo.
- Per accedere ad un membro utilizziamo la sintassi `variabile.membro`

```
#include <stdio.h>

#define LEN 20

struct Studente {
    char nome[LEN];
    int eta;
};

int main(void) {

    struct Studente s= { "Antonio",15};

    printf("Nome: %s, Età: %d\n",s.nome, s.eta);
    printf("Dimensione in memoria: %lu\n", sizeof(s));

    return 0;
}
```

Tipi di dato custom - struct

```
> clang-7 -pthread -lm -o main main.c
main.c:12:28: warning: incompatible pointer to integer conversion
      initializing 'char' with an expression of type 'char [8]'
      [-Wint-conversion]
      struct Studente s= { 15, "Antonio"};
                          ~~~~~
1 warning generated.
> ./main
Nome: T, Età: 0
Dimensione in memoria: 24
> 
```

```
#include <stdio.h>

#define LEN 20

struct Studente {
    char nome[LEN];
    int eta;
};

int main(void) {

    struct Studente s= { 15, "Antonio"};

    printf("Nome: %s, Età: %d\n", s.nome, s.eta);
    printf("Dimensione in memoria: %lu\n", sizeof(s));

    return 0;
}
```

Array bidimensionali

```
int a[3][3];
```

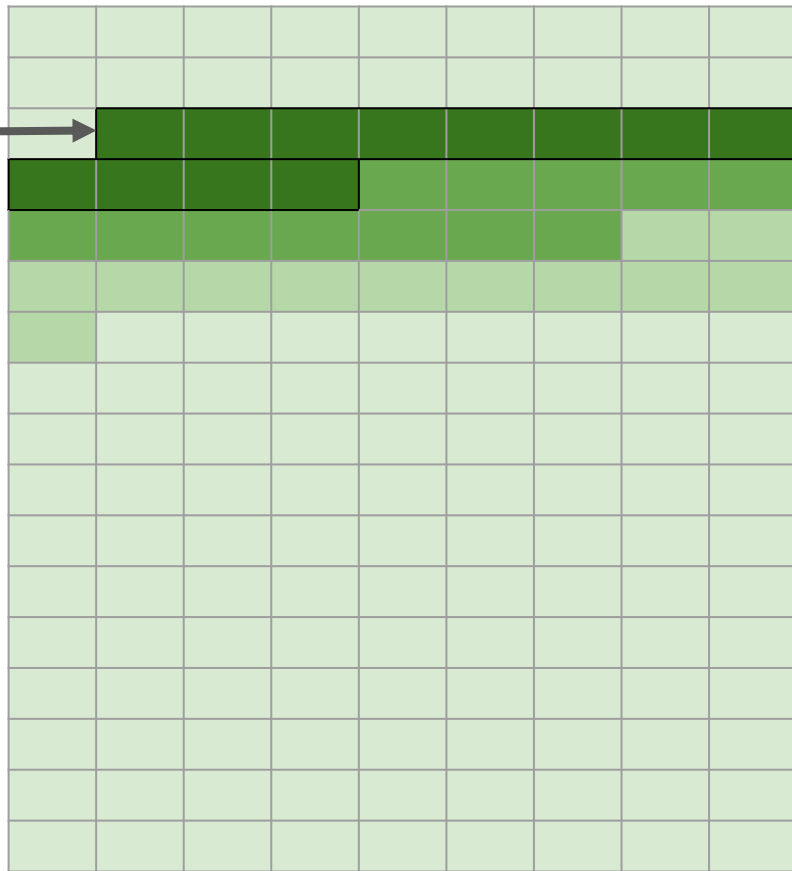
- crea una variabile `a`
 - alloca in memoria sullo stack 9 interi contigui
 - memorizza in `a` l'indirizzo dell'inizio della zona di memoria allocate
 - viene allocata per righe
 - usato per le **matrici**
- La seconda dimensione è molto importante (aiuta il compilatore a calcolare la locazione di memoria giusta, dati gli indici)
 - Per passare un array bidimensionale come parametro dobbiamo specificare la seconda dimensione nella definizione di funzione

```
int f(int[][3]) {}
```

- Si possono definire array con quante dimensioni vogliamo

```
int a[3][4][2];
```

a



Array bidimensionali

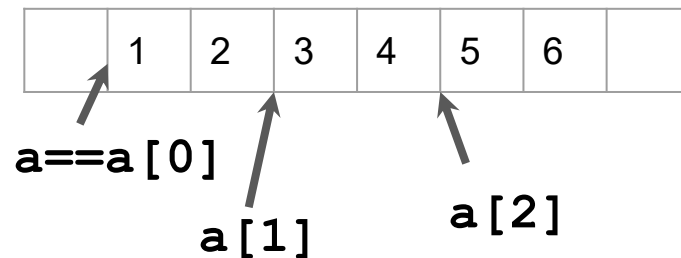
```
#include <stdio.h>

int main(void) {

    int a[3][2]={1,2},{3,4},{5,6}};

    printf("%p\n", a);
    printf("%p, %p, %p\n", a[0], a[1], a[2]);

}
```



0x7ffd2ba5ac10

0x7ffd2ba5ac10, 0x7ffd2ba5ac18, 0x7ffd2ba5ac20

Tipi di dato custom - enum

- `enum` definisce un tipo di dato enumerativo, usato per rendere autodocumentante il codice
- Esempio:
 - `enum giorni {lunedì, martedì, mercoledì, giovedì, venerdì, sabato, domenica};`
- Ogni identificatore è una costante intera che, per default, parte dal valore 0 e incrementa di 1. Si può usare ovunque sia possibile usare una costante di tale tipo. Esempio:

```
enum giorni giorno;  
switch(giorno):  
{  
    case lunedì:  
        printf("lunedì");  
        break;  
    case martedì:  
        printf("martedì");  
        break;  
    ...  
    default:  
        printf("domenica");  
}
```

- Si può cambiare i valori assegnati agli identificatori:
 - `enum giorni {lunedì = 1, martedì, mercoledì, giovedì=7, venerdì, sabato, domenica};`

Tipi di dato custom - typedef

- `typedef` si può usare per rinominare dei tipi esistenti, ad esempio nel caso di nomi di tipo lunghi, o per aggiungere nomi maggiormente mnemonici.
- Si associa ad un identificatore, come nei seguenti casi, e si usa l'identificatore come sostitutivo lessicale al tipo indicato:

```
typedef char[] String;
```

- Le keyword `struct` e `union` vanno sempre ripetute quando si dichiara una nuova variabile.
- E' comodo dare un nome più corto al nuovo tipo di dato:

```
typedef struct {...} Studente;
```

```
typedef enum {...} Giorni;
```

Parametri dalla linea di comando

- Possiamo passare al programma dei parametri e opzioni indicate durante la sua invocazione a linea di comando
 - Simile alla chiamata della shell di comandi UNIX
- Esempio: nel calcolo del max passiamo i due valori di *a* e *b* dalla linea di comando

```
./main 7 5    - stampa 7
```

```
./main 2 3    - stampa 3
```


Parametri dalla linea di comando

Vengono passati al programma tramite parametri della funzione `main`:

```
int main(int argc, char* argv[])
```

`argc` - numero di parametri

`argv` - parametri stessi - stringhe (`char*`)

`argv[0]` - nome del programma

Domande?



Esercizi

- Scrivere un programma che legge n e due vettori di dimensione n , e fa il prodotto scalare tra i due vettori.
- Scrivere un programma che legge n e due matrici quadrate di dimensione $n \times n$, e fa il prodotto scalare tra le due matrici.
- Scrivere un programma che fa la somma dei parametri ricevuti dalla linea di comando

Esercizi

- Scrivere un programma che calcoli il prodotto scalare di un array di interi A, la cui dimensione deve essere chiesta in input all'utente e . Il programma deve invertire l'array senza utilizzare un array di appoggio, ossia scambiare il contenuto della prima e dell'ultima cella, della seconda e della penultima, ecc... ed infine stampare a video ogni singolo elemento dell'array modificato.
- Si scriva un programma di gestione dell'anagrafe dei dipendenti di un'azienda che legga i seguenti dati per 6 diversi dipendenti:
 - Un codice identificativo del dipendente (intero), **diverso** per ogni dipendente
 - L'età del dipendente in anni (un intero)
 - Stipendio (un float, indicato in euro come segue: 16540.32);
 - Il tipo di dipendente, indicando con il numero 0 i dirigenti, 1 i quadri, 2 gli operai e 3 gli impiegati, usando una opportuna `enum` con **etichette** rispettivamente **dirigente**, **quadro**, **operaio** e **impiegato**.

Il numero di dipendenti deve essere chiesto in input all'utente; per ogni dipendente, si chieda in input il codice identificativo, controllando se è diverso da uno già immesso e richiedendolo se è diverso, l'età, lo stipendio e il tipo di dipendente.

Nello stesso ordine in cui sono stati immessi i dati dei dipendenti si stampino, uno per linea, il codice identificativo, l'età e il ruolo di ogni dipendente (l'etichetta) che guadagni meno dello stipendio medio aritmetico di tutti i dipendenti dell'azienda. I valori stampati devono essere separati da un unico spazio.

Esempio di Input:

35,1,56,55650.33, 0, 2, 28, 36540.56, 1, 3, 22, 22120.54, 2, 4, 32, 28450.45, 3, 5, 45, 18960.65, 2, 6, 33,38960.54, 1

Esempio di Output:

3 22 operaio

4 32 impiegato

Esercizi: soluzione 1

```
#include <stdio.h>
int main()
{
    int i,j,scambio;
    int A[10];

    for ( i = 0; i < 10; i++ )
        A[i]=i;

    /* Inversione, scorro l'array fino alla metà della sua dimensione */
    for ( i = 0; i < 10/2; i++ )
    {
        j = (10-1)-i;
        scambio = A[i];
        A[i] = A[j];
        A[j] = scambio;
    }

    /* Output */
    for ( i = 0; i < 10; i++ )
        printf("%d\n", A[i]);
    return 0;
}
```

Esercizi: soluzione 2

```
#include <stdio.h>
#include <stdlib.h>

typedef enum
{
    dirigente,
    quadro,
    operaio,
    impiegato
} ruolo_t;

typedef struct
{
    int id;
    int eta;
    float salario;
    ruolo_t ruolo;
} dipendente_t;

int main(int argc, char *argv[])
{
    int i, id_dip, eta_dip, ruolo_dip, eta;
    float media = 0, salario_dip;

    dipendente_t dipendenti[6];
    scanf("%d",&eta);
```

```
    for (i=0; i<6; i++)
    {
        scanf("%d",&id_dip);
        scanf("%d",&eta_dip);
        scanf("%f",&salario_dip);
        scanf("%d",&ruolo_dip);
        dipendenti[i].id = id_dip;
        dipendenti[i].eta = eta_dip;
        dipendenti[i].salario = salario_dip;
        dipendenti[i].ruolo = ruolo_dip;

        /* calcolo della media, fatto durante l'inserimento */
        media += salario_dip;
    }

    media /= 6;

    for (i=0; i<6; i++)
        if (dipendenti[i].salario < media && dipendenti[i].eta <= eta){
            printf("%d ", dipendenti[i].id);
            printf("%d ", dipendenti[i].eta);
            switch(dipendenti[i].ruolo){
                case dirigente:
                    printf("dirigente\n");
                    break;
                case quadro:
                    printf("quadro\n");
                    break;
                case operaio:
                    printf("operaio\n");
                    break;
                default:
                    printf("impiegato\n");
            }
        }

    return 0;
}
```